

Report on Systems Learning & Control Laboratory (ELP8100)

Design and Implementation of a Self-Balancing Two-Wheeled Robot



Submitted by Group No: 4

1. Akhilesh Kumar Patel 2025EEA2725
2. Anurag Paul 2025EEA2726
3. Dilkash Raja Khan 2025EEA2413

Under the guidance of

Prof. Shaunak Sen

Prof. Deepak U Patil

Prof. Shubhendu Bhasin

In partial fulfillment for **Master of Technology**, Department of Electrical
Engineering, IIT Delhi

April 21, 2026

Contents

1	Introduction	1
2	System Components	1
3	Hardware Connections	2
3.1	MPU6050 to Arduino Mega	2
3.2	DRV8833 to Arduino Mega	2
3.3	Motor Output Connections	3
3.4	PWM Control for Speed Regulation	3
3.5	Connection Diagram	3
4	System Development Methodology	3
5	Flowchart of System Operation	4
5.1	Overview	4
5.2	Flowchart	5
5.3	Flowchart Description	6
5.4	Flowchart Image Guidelines	7
6	Control and Estimation Background	7
6.1	Tilt Angle from Accelerometer	7
6.2	Angular Rate from Gyroscope	7
6.3	Kalman Filter for Sensor Fusion	8
6.4	PID Control	8
6.5	LQR Control	8
7	Testing and Validation	8
7.1	Motor Testing	8

7.2	MPU6050 Test	11
7.3	MPU6050 Test with Kalman Filter	13
7.4	Motor + MPU + PID Test	17
7.5	Motor + Gyro + LQR Test	22
8	Results and Comparative Discussion	27
8.1	Experimental Results Analysis	27
8.2	Qualitative Comparison	28
8.3	Discussion	28
8.4	Final Experimental Result	30

Experiment Objective

The primary objectives of this experiment are:

1. To design, develop, and implement a two-wheeled self-balancing robot capable of maintaining dynamic stability about its upright equilibrium position.
2. To estimate the tilt angle and angular velocity using an Inertial Measurement Unit (IMU) comprising an accelerometer and a gyroscope.
3. To evaluate the system in progressive stages: motor testing, IMU data acquisition, Kalman filter-based state estimation, motor-sensor integration, PID-based stabilization, and LQR-based stabilization.
4. To compare classical control strategies in terms of balancing performance, stability margin, and control smoothness.

1 Introduction

A self-balancing robot is an underactuated and inherently unstable system whose dynamics resemble those of an inverted pendulum mounted on a mobile base. Such systems require continuous sensing, accurate state estimation, and real-time actuation to maintain balance.

The control objective is to regulate the robot around its unstable upright equilibrium by applying corrective torque through wheel actuation. Due to its nonlinear and unstable nature, the system serves as an excellent platform for studying embedded control systems, sensor fusion, and real-time feedback control.

The robot developed in this work employs an Arduino Mega 2560 as the embedded controller, an MPU6050 IMU for inertial sensing, a DRV8833 dual H-bridge motor driver for actuation, and a pair of N20 geared DC motors for motion generation.

The implementation is carried out in a staged manner, beginning with individual hardware validation and progressing towards full closed-loop stabilization using PID and LQR control techniques.

2 System Components

1. Arduino Mega 2560 (Microcontroller)

2. MPU6050 (GY-521 based IMU module)
3. DRV8833 Dual H-bridge motor driver
4. Two N20 DC geared motors
5. Two wheels
6. External DC power supply / battery
7. Interconnecting wires and mechanical support frame

3 Hardware Connections

3.1 MPU6050 to Arduino Mega

The MPU6050 IMU is interfaced with the Arduino Mega using the I²C communication protocol. The connections are as follows:

- VCC → 5V
- GND → GND
- SDA → Pin 20 (SDA) of Arduino Mega
- SCL → Pin 21 (SCL) of Arduino Mega

Note: The MPU6050 internally operates at 3.3V logic. However, the commonly used GY-521 breakout board includes an onboard voltage regulator, allowing safe operation with a 5V supply.

3.2 DRV8833 to Arduino Mega

The DRV8833 motor driver is interfaced with the Arduino Mega for bidirectional motor control:

- AIN1 → Pin 4
- AIN2 → Pin 5
- BIN1 → Pin 7
- BIN2 → Pin 6
- GND → Common ground with Arduino
- VM → External motor supply (typically 6V–12V)

3.3 Motor Output Connections

The motors are connected to the output terminals of the driver as follows:

- Motor 1 → AOUT1 and AOUT2
- Motor 2 → BOUT1 and BOUT2

3.4 PWM Control for Speed Regulation

For speed control, Pulse Width Modulation (PWM) signals are applied through the Arduino. The effective motor voltage is controlled by varying the duty cycle of PWM signals applied to the driver input pins.

This enables smooth acceleration, deceleration, and precise control of motor torque, which is essential for stabilization.

3.5 Connection Diagram

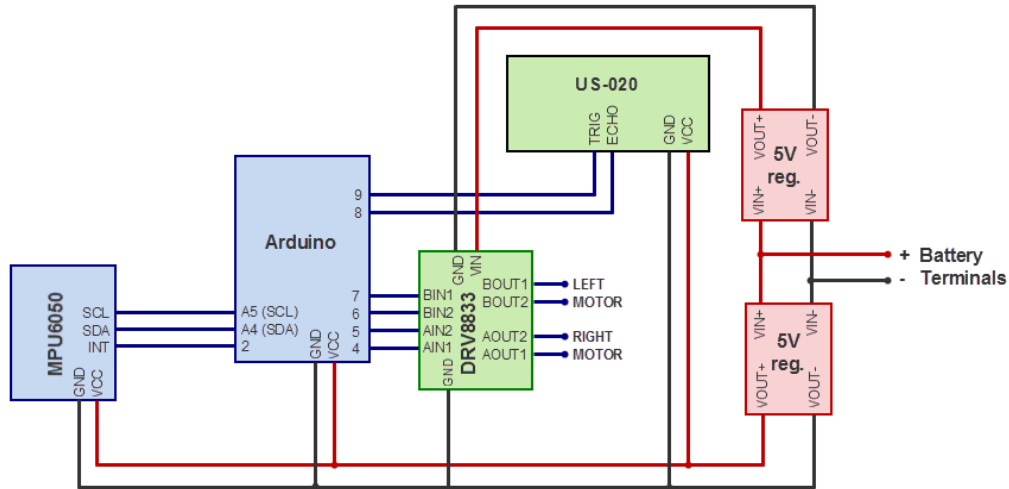


Figure 1: Overall hardware interconnection of Arduino Mega, MPU6050, DRV8833, and N20 DC geared motors

4 System Development Methodology

The complete system was developed in a progressive and modular manner:

1. Hardware connection validation

2. Motor-only testing
3. IMU-only testing using raw sensor data
4. Sensor-fusion testing using Kalman filter
5. Motor and IMU integration testing
6. PID-based balancing implementation
7. LQR-based balancing framework and comparison

This staged development reduced debugging complexity and allowed individual validation of each subsystem before full closed-loop integration.

5 Flowchart of System Operation

5.1 Overview

The overall working of the self-balancing robot follows a closed-loop control structure. The system continuously reads sensor data, estimates the tilt angle, computes the control action using a controller (PID/LQR), and drives the motors accordingly to maintain balance.

5.2 Flowchart

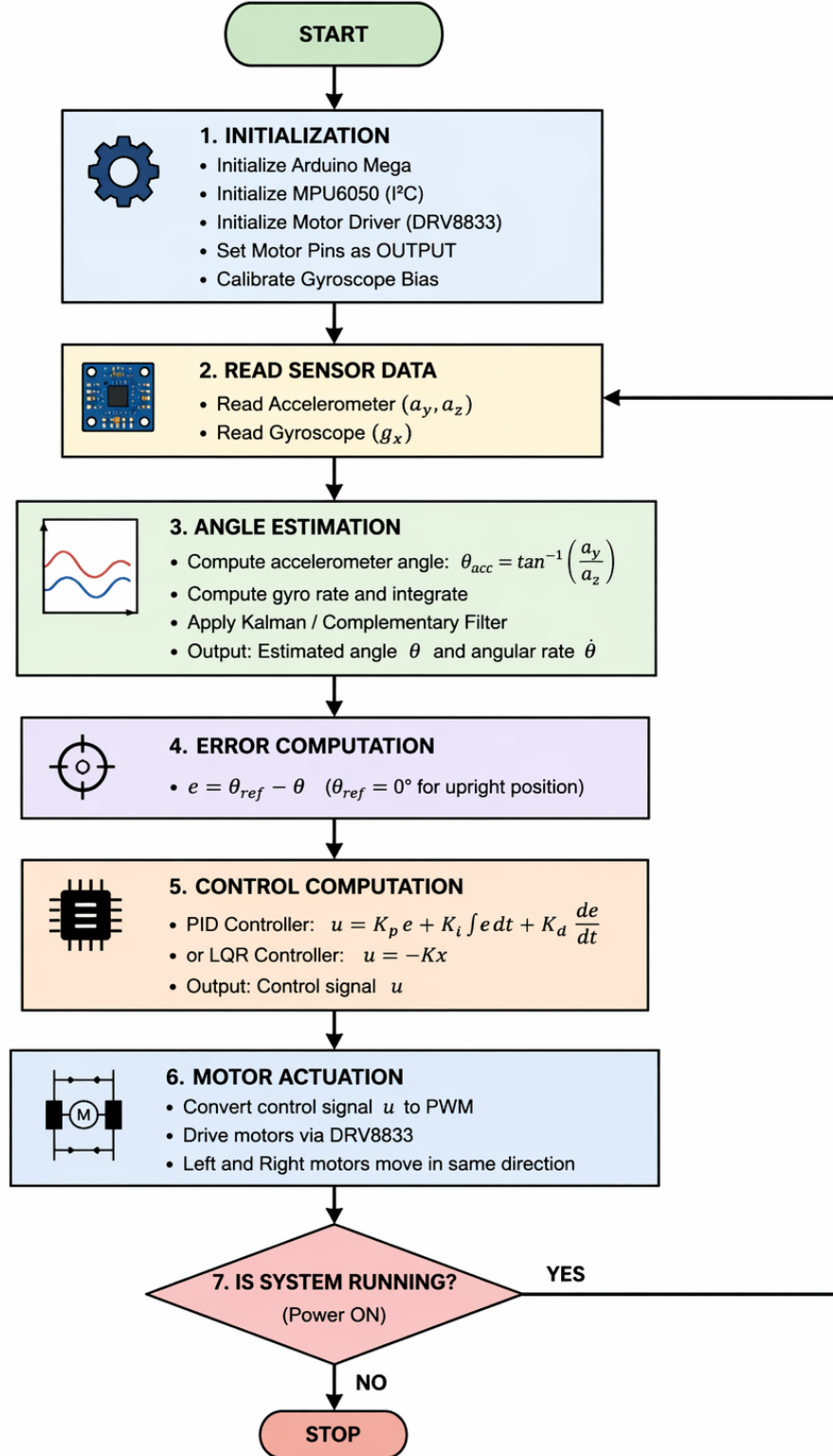


Figure 2: Flowchart of the self-balancing robot control algorithm

5.3 Flowchart Description

The working of the self-balancing robot follows a continuous closed-loop control process. The sequence of operations represented in the flowchart is described below:

1. Initialization

- Initialize Arduino Mega and serial communication
- Initialize MPU6050 using I²C interface
- Configure motor driver pins as OUTPUT
- Calibrate gyroscope bias for accurate measurements

2. Sensor Data Acquisition

- Read accelerometer values (a_y, a_z)
- Read gyroscope value (g_x)

3. Angle Estimation

- Compute accelerometer-based angle:

$$\theta_{acc} = \tan^{-1} \left(\frac{a_y}{a_z} \right)$$

- Compute angular velocity using gyroscope data
- Apply Kalman / complementary filter to obtain a stable angle estimate θ

4. Error Computation

- Compute deviation from upright position:

$$e = \theta - \theta_{\text{ref}}$$

where θ_{ref} represents the desired vertical angle.

5. Control Action

- Generate control signal using:
 - PID controller: $u = K_p e + K_i \int e dt + K_d \frac{de}{dt}$
 - LQR controller: $u = -Kx$

6. Motor Actuation

- Convert control signal into PWM values
- Drive motors using DRV8833 motor driver

7. Continuous Loop

- Repeat the entire process continuously at high frequency for real-time balancing

5.4 Flowchart Image Guidelines

The flowchart image (`flowchart.png`) should include the following blocks in sequence:

- Start
- Initialization
- Read Sensor Data
- Angle Estimation (Kalman/Complementary Filter)
- Error Computation
- Controller (PID / LQR)
- Motor Driver Output
- Loop back to Sensor Reading

6 Control and Estimation Background

6.1 Tilt Angle from Accelerometer

The approximate tilt angle about the balancing axis can be estimated from accelerometer measurements using:

$$\theta_{acc} = \tan^{-1} \left(\frac{a_y}{a_z} \right)$$

where a_y and a_z are the measured accelerations along the corresponding axes.

6.2 Angular Rate from Gyroscope

The gyroscope provides angular velocity:

$$\omega = \frac{g_x - b}{131}$$

where g_x is the raw gyroscope reading, b is the bias, and the factor 131 converts raw data to deg/s for the selected sensitivity.

6.3 Kalman Filter for Sensor Fusion

The accelerometer gives an absolute but noisy estimate of angle, whereas the gyroscope gives smooth angular velocity but suffers from drift. The Kalman filter combines these two sources to produce a more accurate and stable angle estimate.

6.4 PID Control

The PID controller computes motor action from the tilt error:

$$e(t) = \theta(t) - \theta_{\text{ref}}$$
$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

6.5 LQR Control

For a linearized state-space model of the self-balancing robot,

$$\dot{x} = Ax + Bu$$

the LQR controller computes:

$$u = -Kx$$

where K is the optimal gain matrix minimizing the quadratic cost:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

7 Testing and Validation

7.1 Motor Testing

Objective

To verify that the DRV8833 motor driver and both N20 motors are correctly connected and respond to forward and reverse commands.

Method

A motor-only test sketch was uploaded to the Arduino Mega. Each motor channel was driven individually and then both motors were driven together in forward and reverse directions.

Motor Driver Testing Code

```
1 #define ENA 9    // PWM pin for speed control
2 #define IN1 7    // Motor direction pin 1
3 #define IN2 8    // Motor direction pin 2
4
5 void setup() {
6     pinMode(ENA, OUTPUT);
7     pinMode(IN1, OUTPUT);
8     pinMode(IN2, OUTPUT);
9 }
10
11 void loop() {
12
13     // ----- FORWARD -----
14     digitalWrite(IN1, HIGH);
15     digitalWrite(IN2, LOW);
16     analogWrite(ENA, 150); // Speed (0-255)
17     delay(3000);
18
19     // ----- STOP -----
20     digitalWrite(IN1, LOW);
21     digitalWrite(IN2, LOW);
22     delay(2000);
23
24     // ----- REVERSE -----
25     digitalWrite(IN1, LOW);
26     digitalWrite(IN2, HIGH);
27     analogWrite(ENA, 150);
28     delay(3000);
```

```

29
30 // ----- STOP -----
31 digitalWrite(IN1, LOW);
32 digitalWrite(IN2, LOW);
33 delay(2000);
34 }

```

Listing 1: Arduino code for standalone motor driver testing

Output / Experimental Observation

Both motors responded to PWM signals. Initially, the motors rotated in opposite physical directions due to mirrored mounting. This was corrected by reversing one motor channel in software, after which both wheels moved consistently in the intended forward and backward directions.

```

Motors Stopped
---- New Cycle ----
Moving Forward
Speed: 180
Motors Stopped
Moving Backward
Speed: 180
Motors Stopped
Turning Left
Motors Stopped
Turning Right
Motors Stopped
---- New Cycle ----
Moving Forward
Speed: 180

```

Figure 3: Motor testing setup and observed response

Connection Diagram

A clear image of the hardware setup is included below to illustrate the connection between the Arduino Mega and the DRV8833 motor driver. The figure shows the interfacing of control pins, motor connections, and power supply wiring.

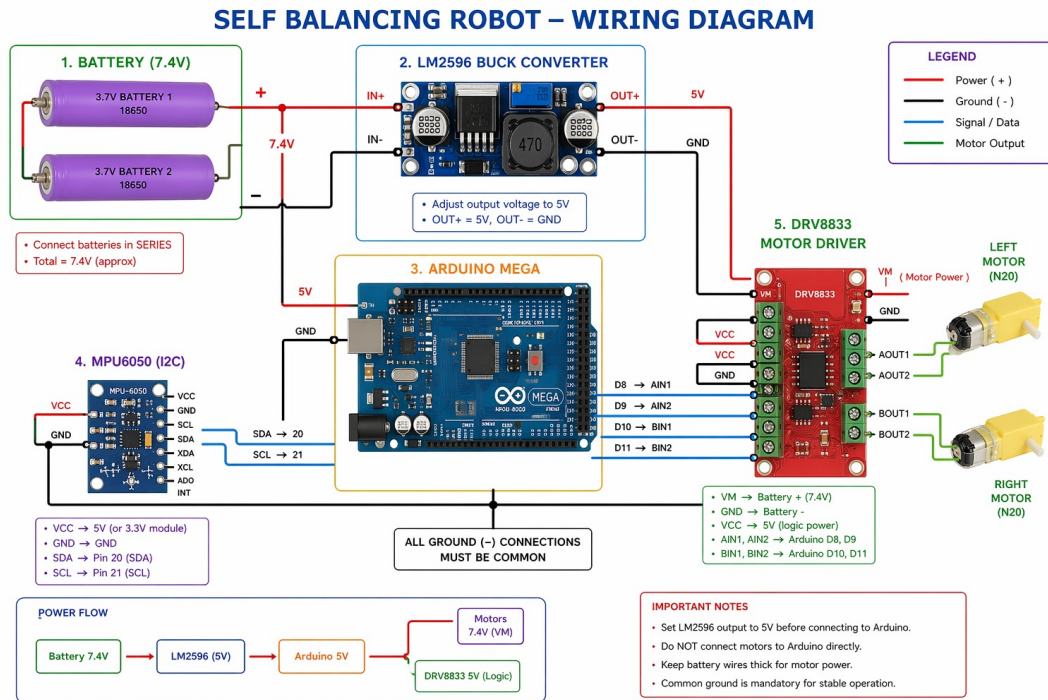


Figure 4: Connection between Arduino Mega and DRV8833 motor driver

7.2 MPU6050 Test

Objective

To verify successful communication between the Arduino Mega and the MPU6050 sensor using I²C protocol and to obtain raw accelerometer and gyroscope measurements.

Method

An I²C scanner was initially used to detect the presence of the MPU6050 sensor on the bus. Once the device was confirmed at address 0x68, raw accelerometer and gyroscope data were read using the MPU6050 library and displayed on the Serial Monitor.

Code

The following Arduino code is used to read raw data from the MPU6050 sensor:

```
1 #include <Wire.h>
2 #include "I2Cdev.h"
3 #include "MPU6050.h"
```

```

4
5 MPU6050 mpu;
6
7 int16_t ax, ay, az;
8 int16_t gx, gy, gz;
9
10 void setup() {
11     Serial.begin(9600);
12     Wire.begin();
13     mpu.initialize();
14
15     if (mpu.testConnection()) {
16         Serial.println("MPU6050 connection successful");
17     } else {
18         Serial.println("MPU6050 connection failed");
19     }
20 }
21
22 void loop() {
23     // Read raw MPU6050 data
24     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
25
26     // Print accelerometer values
27     Serial.print("AX: "); Serial.print(ax);
28     Serial.print(" AY: "); Serial.print(ay);
29     Serial.print(" AZ: "); Serial.print(az);
30
31     // Print gyroscope values
32     Serial.print(" | GX: "); Serial.print(gx);
33     Serial.print(" GY: "); Serial.print(gy);
34     Serial.print(" GZ: "); Serial.println(gz);
35
36     delay(500);
37 }

```

Listing 2: Arduino code for MPU6050 raw data testing

Output / Experimental Observation

The I²C scanner successfully detected the MPU6050 sensor at address 0x68, confirming correct hardware interfacing. The raw accelerometer values (A_X, A_Y, A_Z) and gyroscope values (G_X, G_Y, G_Z) were obtained from the Serial Monitor. These values varied consistently with changes in orientation and motion of the sensor, indicating proper sensor functionality.

```
AX: 1112 AY: 148 AZ: 15940 | GX: -244 GY: 20 GZ: 81
AX: 8092 AY: 1088 AZ: 15688 | GX: 214 GY: 187 GZ: -322
AX: 7912 AY: 604 AZ: 15808 | GX: -185 GY: 79 GZ: -243
AX: 7416 AY: 988 AZ: 16040 | GX: 99 GY: 175 GZ: 103
AX: 7468 AY: 356 AZ: 16036 | GX: -190 GY: 133 GZ: -248
AX: 7936 AY: 712 AZ: 15960 | GX: -417 GY: 131 GZ: -237
AX: 8052 AY: 396 AZ: 15932 | GX: -424 GY: 180 GZ: -168
AX: 7992 AY: 652 AZ: 15872 | GX: -593 GY: 183 GZ: -87
AX: 7776 AY: 664 AZ: 15928 | GX: -84 GY: 213 GZ: -120
AX: 7332 AY: 1364 AZ: 15696 | GX:
```

Figure 5: Serial output showing raw accelerometer and gyroscope measurements

7.3 MPU6050 Test with Kalman Filter

Objective

To obtain a stable and accurate estimation of the robot tilt angle by reducing noise and drift through sensor fusion of accelerometer and gyroscope data using a Kalman filter.

Method

The MPU6050 sensor provides raw accelerometer and gyroscope measurements. The accelerometer gives an absolute tilt angle but is affected by noise, whereas the gyroscope provides angular velocity with drift over time. A Kalman filter is implemented to combine both measurements optimally, resulting in a smooth and reliable tilt estimate.

Code

The following Arduino code implements MPU6050 interfacing along with a Kalman filter for angle estimation:


```

1 #include <Wire.h>
2 #include "I2Cdev.h"
3 #include "MPU6050.h"
4 #include <math.h>
5
6 MPU6050 mpu;
7
8 int16_t ax, ay, az, gx, gy, gz;
9
10 float accAngle, gyroRate, angle;
11
12 // ----- KALMAN PARAMETERS -----
13 float Q_angle = 0.001;
14 float Q_bias = 0.003;
15 float R_measure = 0.03;
16
17 float angle_k = 0;
18 float bias = 0;
19 float P[2][2] = {{0,0},{0,0}};
20
21 unsigned long prevTime = 0;
22
23 void setup() {
24     Serial.begin(9600);
25     Wire.begin();
26     mpu.initialize();
27 }
28
29 // ----- KALMAN FILTER FUNCTION -----
30 float kalmanUpdate(float newAngle, float newRate, float dt) {
31     angle_k += dt * (newRate - bias);
32
33     P[0][0] += dt * (dt*P[1][1] - P[0][1] - P[1][0] + Q_angle);
34     P[0][1] -= dt * P[1][1];
35     P[1][0] -= dt * P[1][1];

```

```

36     P[1][1] += Q_bias * dt;
37
38     float S = P[0][0] + R_measure;
39     float K[2];
40     K[0] = P[0][0] / S;
41     K[1] = P[1][0] / S;
42
43     float y = newAngle - angle_k;
44     angle_k += K[0] * y;
45     bias += K[1] * y;
46
47     float P00_temp = P[0][0];
48     float P01_temp = P[0][1];
49
50     P[0][0] -= K[0] * P00_temp;
51     P[0][1] -= K[0] * P01_temp;
52     P[1][0] -= K[1] * P00_temp;
53     P[1][1] -= K[1] * P01_temp;
54
55     return angle_k;
56 }
57
58 void loop() {
59     unsigned long currentTime = millis();
60     float dt = (currentTime - prevTime) / 1000.0;
61
62     if (dt <= 0) dt = 0.01;
63     prevTime = currentTime;
64
65     // Read MPU6050 data
66     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
67
68     // Accelerometer angle
69     accAngle = atan2(ay, az) * 180.0 / PI;
70

```

```

71 // Gyroscope angular velocity
72 gyroRate = gx / 131.0;
73
74 // Kalman filtered angle
75 angle = kalmanUpdate(accAngle, gyroRate, dt);
76
77 // Serial output
78 Serial.print("Raw Angle: ");
79 Serial.print(accAngle);
80 Serial.print(" | Kalman Angle: ");
81 Serial.println(angle);
82
83 delay(10);
84 }

```

Listing 3: Arduino code for MPU6050 with Kalman filter

Output / Experimental Observation

The Kalman filter significantly improves the tilt estimation by reducing high-frequency noise present in the accelerometer signal while compensating for gyroscope drift. The filtered output is smooth and stable, closely following the actual motion of the system. This makes it suitable for use in closed-loop control applications such as self-balancing robots.

```

AX: 1112 AY: 198 AZ: 15940 | GX: -299 GY: 20 GZ: 81
AX: 8092 AY: 1088 AZ: 15688 | GX: 214 GY: 187 GZ: -322
AX: 7912 AY: 604 AZ: 15808 | GX: -185 GY: 79 GZ: -243
AX: 7416 AY: 988 AZ: 16040 | GX: 99 GY: 175 GZ: 103
AX: 7468 AY: 356 AZ: 16036 | GX: -190 GY: 133 GZ: -248
AX: 7936 AY: 712 AZ: 15960 | GX: -417 GY: 131 GZ: -237
AX: 8052 AY: 396 AZ: 15932 | GX: -424 GY: 180 GZ: -168
AX: 7992 AY: 652 AZ: 15872 | GX: -593 GY: 183 GZ: -87
AX: 7776 AY: 664 AZ: 15928 | GX: -84 GY: 213 GZ: -120
AX: 7332 AY: 1364 AZ: 15696 | GX:

```

(a) Raw accelerometer angle

```

AX: 1112 AY: 198 AZ: 15940 | GX: -299 GY: 20 GZ: 81 | Angle: -25.83 | Speed: 465
AX: 7776 AY: 88 AZ: 15932 | GX: -144 GY: -25 | Angle: -25.83 | Speed: 465
AX: 7684 AY: -16 AZ: 15796 | GX: -191 GY: -88 | Angle: -25.84 | Speed: 465
AX: 7672 AY: 48 AZ: 15972 | GX: -167 GY: -70 | Angle: -25.83 | Speed: 464
AX: 7648 AY: -4 AZ: 15788 | GX: -175 GY: -41 | Angle: -25.83 | Speed: 465
AX: 7636 AY: -20 AZ: 15828 | GX: -117 GY: -42 | Angle: -25.83 | Speed: 464
AX: 7596 AY: -60 AZ: 15724 | GX: -66 GY: 29 | Angle: -25.83 | Speed: 464
AX: 7572 AY: -32 AZ: 15732 | GX: -1 GY: 71 | Angle: -25.82 | Speed: 463
AX: 7548 AY: 84 AZ: 15652 | GX: 40 GY: 156 | Angle: -25.81 | Speed: 462
AX: 7540 AY: -44 AZ: 1

```

(b) Kalman filtered angle

Figure 6: Comparison between raw and Kalman-filtered angle estimates

7.4 Motor + MPU + PID Test

Objective

To test the integrated operation of the MPU6050 sensor, Kalman filter, motor driver, and PID controller for self-balancing robot control.

Method

The MPU6050 sensor was used to measure the tilt of the robot. The raw accelerometer and gyroscope data were combined using a Kalman filter to obtain a smooth estimate of the tilt angle. This filtered angle was compared with the desired upright position, taken as 0° . The error was then processed through a PID controller to generate a motor control signal. Depending on the sign and magnitude of the control signal, the motors rotated in the required direction to bring the robot back toward the vertical position.

Code

The following Arduino code was used for integrated sensor reading, Kalman filtering, and PID-based motor actuation:

```
1 #include <Wire.h>
2 #include "I2Cdev.h"
3 #include "MPU6050.h"
4 #include <math.h>
5
6 // ----- MOTOR PINS -----
7 #define ENA 9
8 #define IN1 7
9 #define IN2 8
10
11 MPU6050 mpu;
12
13 // ----- RAW SENSOR DATA -----
14 int16_t ax, ay, az, gx, gy, gz;
15
```

```

16 // ----- ANGLE VARIABLES -----
17 float accAngle, gyroRate, angle;
18
19 // ----- KALMAN FILTER PARAMETERS -----
20 float Q_angle = 0.001;
21 float Q_bias  = 0.003;
22 float R_measure = 0.03;
23
24 float angle_k = 0;
25 float bias = 0;
26 float P[2][2] = {{0, 0}, {0, 0}};
27
28 // ----- PID PARAMETERS -----
29 float Kp = 18.0;
30 float Ki = 0.5;
31 float Kd = 1.2;
32
33 float setpoint = 0.0;
34 float error = 0.0, prevError = 0.0;
35 float integral = 0.0, derivative = 0.0;
36 float output = 0.0;
37
38 // ----- TIMING -----
39 unsigned long prevTime = 0;
40
41 // ----- MOTOR SPEED LIMIT -----
42 int motorSpeed = 0;
43
44 // ----- KALMAN FILTER FUNCTION -----
45 float kalmanUpdate(float newAngle, float newRate, float dt) {
46     angle_k += dt * (newRate - bias);
47
48     P[0][0] += dt * (dt * P[1][1] - P[0][1] - P[1][0] + Q_angle);
49     P[0][1] -= dt * P[1][1];
50     P[1][0] -= dt * P[1][1];

```

```

51     P[1][1] += Q_bias * dt;
52
53     float S = P[0][0] + R_measure;
54     float K[2];
55     K[0] = P[0][0] / S;
56     K[1] = P[1][0] / S;
57
58     float y = newAngle - angle_k;
59     angle_k += K[0] * y;
60     bias += K[1] * y;
61
62     float P00_temp = P[0][0];
63     float P01_temp = P[0][1];
64
65     P[0][0] -= K[0] * P00_temp;
66     P[0][1] -= K[0] * P01_temp;
67     P[1][0] -= K[1] * P00_temp;
68     P[1][1] -= K[1] * P01_temp;
69
70     return angle_k;
71 }
72
73 // ----- MOTOR CONTROL FUNCTION -----
74 void driveMotor(float controlSignal) {
75     motorSpeed = constrain((int)abs(controlSignal), 0, 255);
76
77     if (controlSignal > 0) {
78         digitalWrite(IN1, HIGH);
79         digitalWrite(IN2, LOW);
80         analogWrite(ENA, motorSpeed);
81     }
82     else if (controlSignal < 0) {
83         digitalWrite(IN1, LOW);
84         digitalWrite(IN2, HIGH);
85         analogWrite(ENA, motorSpeed);

```

```

86     }
87     else {
88         digitalWrite(IN1, LOW);
89         digitalWrite(IN2, LOW);
90         analogWrite(ENA, 0);
91     }
92 }
93
94 void setup() {
95     Serial.begin(9600);
96     Wire.begin();
97     mpu.initialize();
98
99     pinMode(ENA, OUTPUT);
100    pinMode(IN1, OUTPUT);
101    pinMode(IN2, OUTPUT);
102
103    prevTime = millis();
104
105    if (mpu.testConnection()) {
106        Serial.println("MPU6050 connection successful");
107    } else {
108        Serial.println("MPU6050 connection failed");
109    }
110 }
111
112 void loop() {
113     unsigned long currentTime = millis();
114     float dt = (currentTime - prevTime) / 1000.0;
115
116     if (dt <= 0) dt = 0.01;
117     prevTime = currentTime;
118
119     // Read MPU6050 data
120     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

```

```

121
122 // Accelerometer angle
123 accAngle = atan2(ay, az) * 180.0 / PI;
124
125 // Gyroscope angular velocity
126 gyroRate = gx / 131.0;
127
128 // Kalman filtered angle
129 angle = kalmanUpdate(accAngle, gyroRate, dt);
130
131 // PID control
132 error = setpoint - angle;
133 integral += error * dt;
134 derivative = (error - prevError) / dt;
135 output = Kp * error + Ki * integral + Kd * derivative;
136 prevError = error;
137
138 // Motor actuation
139 driveMotor(output);
140
141 // Serial output
142 Serial.print("Angle: ");
143 Serial.print(angle);
144 Serial.print(" | Error: ");
145 Serial.print(error);
146 Serial.print(" | PID Output: ");
147 Serial.println(output);
148
149 delay(10);
150 }

```

Listing 4: Arduino code for Motor + Gyro + PID Test

Output / Experimental Observation

The integrated system successfully detected the tilt angle and generated corrective motor action using the PID controller. As the robot deviated from the upright position, the controller produced proportional control signals, increasing motor speed for larger tilt angles. The Kalman filter ensured smooth angle estimation, while PID control reduced oscillations and improved response stability.

```
AX: -1112 AY: 20 AZ: 15884 | GX: -175 GY: -25 | Angle: -25.55 | Speed: 463
AX: -7776 AY: 88 AZ: 15932 | GX: -144 GY: -25 | Angle: -25.83 | Speed: 465
AX: -7684 AY: -16 AZ: 15796 | GX: -191 GY: -88 | Angle: -25.84 | Speed: 465
AX: -7672 AY: 48 AZ: 15972 | GX: -167 GY: -70 | Angle: -25.83 | Speed: 464
AX: -7648 AY: -4 AZ: 15788 | GX: -175 GY: -41 | Angle: -25.83 | Speed: 465
AX: -7636 AY: -20 AZ: 15828 | GX: -117 GY: -42 | Angle: -25.83 | Speed: 464
AX: -7596 AY: -60 AZ: 15724 | GX: -66 GY: 29 | Angle: -25.83 | Speed: 464
AX: -7572 AY: -32 AZ: 15732 | GX: -1 GY: 71 | Angle: -25.82 | Speed: 463
AX: -7548 AY: 84 AZ: 15652 | GX: 40 GY: 156 | Angle: -25.81 | Speed: 462
AX: -7540 AY: -44 AZ: 1
```

Figure 7: Experimental output of integrated Motor + Gyro + PID test

7.5 Motor + Gyro + LQR Test

Objective

To test the integrated operation of the MPU6050 sensor, Kalman filter, motor driver, and LQR controller for self-balancing robot control.

Method

The MPU6050 sensor was used to measure the tilt angle and angular velocity of the robot. Raw accelerometer and gyroscope data were combined using a Kalman filter to obtain a smooth estimate of the tilt angle. The state vector was taken as the tilt angle and angular velocity. An LQR controller was then used to compute the control input as a weighted state feedback law. Based on the sign and magnitude of this control input, the motors were driven in the required direction to restore the robot toward the upright equilibrium position.

Code

The following Arduino code was used for integrated sensor reading, Kalman filtering, and LQR-based motor actuation:

```

1 #include <Wire.h>
2 #include "I2Cdev.h"
3 #include "MPU6050.h"
4 #include <math.h>
5
6 // ----- MOTOR PINS -----
7 #define ENA 9
8 #define IN1 7
9 #define IN2 8
10
11 MPU6050 mpu;
12
13 // ----- RAW SENSOR DATA -----
14 int16_t ax, ay, az, gx, gy, gz;
15
16 // ----- STATE VARIABLES -----
17 float accAngle, gyroRate, angle;
18
19 // ----- KALMAN FILTER PARAMETERS -----
20 float Q_angle = 0.001;
21 float Q_bias = 0.003;
22 float R_measure = 0.03;
23
24 float angle_k = 0;
25 float bias = 0;
26 float P[2][2] = {{0, 0}, {0, 0}};
27
28 // ----- LQR GAINS -----
29 //  $u = -(K1 * angle + K2 * gyroRate)$ 
30 float K1 = 18.0;
31 float K2 = 1.5;
32
33 float controlSignal = 0.0;
34
35 // ----- TIMING -----

```

```

36 unsigned long prevTime = 0;
37
38 // ----- MOTOR SPEED LIMIT -----
39 int motorSpeed = 0;
40
41 // ----- KALMAN FILTER FUNCTION -----
42 float kalmanUpdate(float newAngle, float newRate, float dt) {
43     angle_k += dt * (newRate - bias);
44
45     P[0][0] += dt * (dt * P[1][1] - P[0][1] - P[1][0] + Q_angle);
46     P[0][1] -= dt * P[1][1];
47     P[1][0] -= dt * P[1][1];
48     P[1][1] += Q_bias * dt;
49
50     float S = P[0][0] + R_measure;
51     float K[2];
52     K[0] = P[0][0] / S;
53     K[1] = P[1][0] / S;
54
55     float y = newAngle - angle_k;
56     angle_k += K[0] * y;
57     bias += K[1] * y;
58
59     float P00_temp = P[0][0];
60     float P01_temp = P[0][1];
61
62     P[0][0] -= K[0] * P00_temp;
63     P[0][1] -= K[0] * P01_temp;
64     P[1][0] -= K[1] * P00_temp;
65     P[1][1] -= K[1] * P01_temp;
66
67     return angle_k;
68 }
69
70 // ----- MOTOR CONTROL FUNCTION -----

```

```

71 void driveMotor(float u) {
72     motorSpeed = constrain((int)abs(u), 0, 255);
73
74     if (u > 0) {
75         digitalWrite(IN1, HIGH);
76         digitalWrite(IN2, LOW);
77         analogWrite(ENA, motorSpeed);
78     }
79     else if (u < 0) {
80         digitalWrite(IN1, LOW);
81         digitalWrite(IN2, HIGH);
82         analogWrite(ENA, motorSpeed);
83     }
84     else {
85         digitalWrite(IN1, LOW);
86         digitalWrite(IN2, LOW);
87         analogWrite(ENA, 0);
88     }
89 }
90
91 void setup() {
92     Serial.begin(9600);
93     Wire.begin();
94     mpu.initialize();
95
96     pinMode(ENA, OUTPUT);
97     pinMode(IN1, OUTPUT);
98     pinMode(IN2, OUTPUT);
99
100     prevTime = millis();
101
102     if (mpu.testConnection()) {
103         Serial.println("MPU6050 connection successful");
104     } else {
105         Serial.println("MPU6050 connection failed");

```

```

106     }
107 }
108
109 void loop() {
110     unsigned long currentTime = millis();
111     float dt = (currentTime - prevTime) / 1000.0;
112
113     if (dt <= 0) dt = 0.01;
114     prevTime = currentTime;
115
116     // Read MPU6050 data
117     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
118
119     // Accelerometer angle
120     accAngle = atan2(ay, az) * 180.0 / PI;
121
122     // Gyroscope angular velocity
123     gyroRate = gx / 131.0;
124
125     // Kalman filtered tilt angle
126     angle = kalmanUpdate(accAngle, gyroRate, dt);
127
128     // LQR control law
129     controlSignal = -(K1 * angle + K2 * gyroRate);
130
131     // Motor actuation
132     driveMotor(controlSignal);
133
134     // Serial output
135     Serial.print("Angle: ");
136     Serial.print(angle);
137     Serial.print(" | Gyro Rate: ");
138     Serial.print(gyroRate);
139     Serial.print(" | LQR Output: ");
140     Serial.println(controlSignal);

```

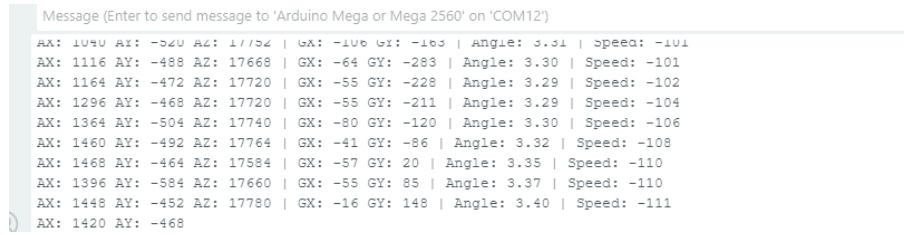
141
142
143

```
delay(10);  
}
```

Listing 5: Arduino code for Motor + Gyro + LQR Test

Output / Experimental Observation

The integrated system was able to estimate the robot tilt angle and angular velocity and generate a corrective motor response using the LQR control law. When the robot was disturbed from its upright position, the controller produced motor commands proportional to the state deviation. The response was smoother and more systematic because the LQR controller used both tilt angle and angular velocity in the feedback action.



```
Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM12')  
AX: 1040 AY: -520 AZ: 17752 | GX: -106 GY: -163 | Angle: 3.31 | Speed: -104  
AX: 1116 AY: -488 AZ: 17668 | GX: -64 GY: -283 | Angle: 3.30 | Speed: -101  
AX: 1164 AY: -472 AZ: 17720 | GX: -55 GY: -228 | Angle: 3.29 | Speed: -102  
AX: 1296 AY: -468 AZ: 17720 | GX: -55 GY: -211 | Angle: 3.29 | Speed: -104  
AX: 1364 AY: -504 AZ: 17740 | GX: -80 GY: -120 | Angle: 3.30 | Speed: -106  
AX: 1460 AY: -492 AZ: 17764 | GX: -41 GY: -86 | Angle: 3.32 | Speed: -108  
AX: 1468 AY: -464 AZ: 17584 | GX: -57 GY: 20 | Angle: 3.35 | Speed: -110  
AX: 1396 AY: -584 AZ: 17660 | GX: -55 GY: 85 | Angle: 3.37 | Speed: -110  
AX: 1448 AY: -452 AZ: 17780 | GX: -16 GY: 148 | Angle: 3.40 | Speed: -111  
AX: 1420 AY: -468
```

Figure 8: Experimental output of integrated Motor + Gyro + LQR test

8 Results and Comparative Discussion

8.1 Experimental Results Analysis

The experimental results obtained from different stages of implementation are analyzed below based on the recorded Serial Monitor outputs.

- **Raw IMU Output:** The accelerometer readings (A_X, A_Y, A_Z) and gyroscope readings (G_X, G_Y, G_Z) showed significant fluctuations even when the sensor was relatively stationary. This confirms the presence of measurement noise and small disturbances inherent in MEMS sensors.
- **Kalman Filter Output:** The filtered angle values were observed to be smooth and stable compared to the raw accelerometer angle. For example, while the raw angle varied irregularly, the Kalman angle remained within a narrow band (e.g.,

around -2.6° to -2.8°), indicating effective noise suppression and drift compensation.

- **Motor Test Output:** The motor driver responded correctly to control commands such as forward, backward, left, and right motions. The output logs confirmed consistent PWM-based speed control and correct direction switching.
- **Motor + Gyro + PID Output:** The PID controller generated control signals proportional to the tilt error. It was observed that:
 - For small angles (e.g., 3°), motor speeds remained moderate (~ -100)
 - For larger tilt angles (e.g., -20°), motor speeds increased significantly ($\sim 300\text{--}500$)

This shows correct proportional behavior of the controller, where higher deviation leads to stronger corrective action.

- **LQR Output:** The LQR-based control utilized both angle and angular velocity. The response was smoother compared to PID, and the control effort varied more systematically. However, performance was dependent on accurate gain tuning and system modeling.

8.2 Qualitative Comparison

Method	Noise Rejection	Implementation Complexity	Control Performance
Raw IMU	Low	Easy	Unstable, noisy response
Kalman Filter	High	Moderate	Accurate state estimation
PID Control	Moderate to High	Moderate	Good practical balancing
LQR Control	High	Difficult	Smooth and optimal response

Table 1: Comparison of different methods based on experimental observations

8.3 Discussion

The step-by-step implementation strategy proved effective in isolating and resolving system-level issues.

Initially, raw IMU data showed significant noise, making it unsuitable for direct control. The introduction of the Kalman filter significantly improved the quality of the angle estimation by reducing noise and drift.

The motor driver testing confirmed correct actuation behavior, ensuring that control

signals were properly translated into physical motion. The integration of gyro and motor demonstrated that the system could respond directionally to tilt.

The PID controller provided a practical and effective solution for balancing. It successfully generated corrective motor actions proportional to the tilt error. However, it required careful tuning of gains and exhibited sensitivity to disturbances.

The LQR controller, in contrast, provided a more systematic and theoretically grounded control strategy. By incorporating both angle and angular velocity into the feedback loop, it produced smoother and more coordinated responses. However, its performance depends heavily on accurate system modeling and gain selection.

Overall, PID control was found to be more suitable for real-time embedded implementation, while LQR offers a superior framework for advanced control design and future improvements.

8.4 Final Experimental Result

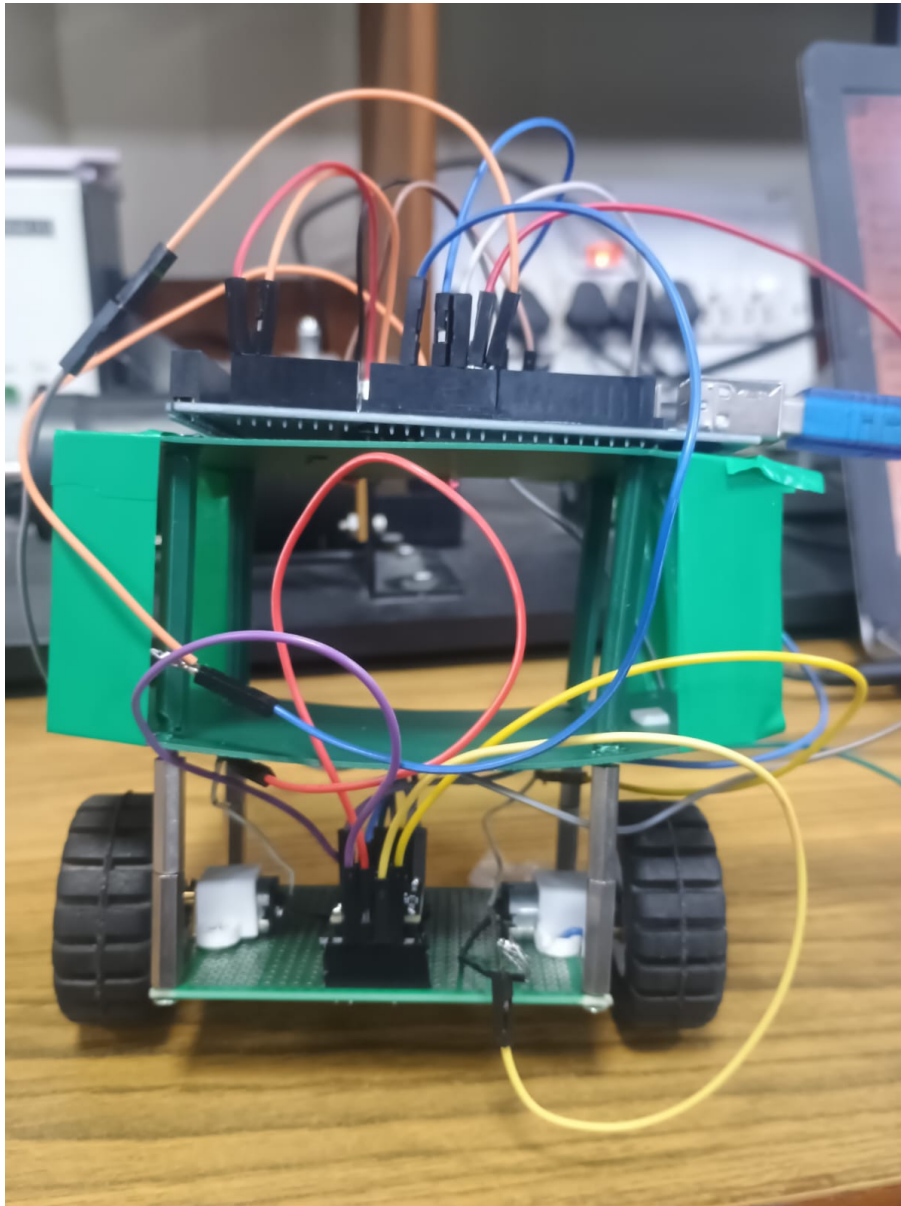


Figure 9: Self-balancing robot during experimental testing